

GraphTARIF: Linear Graph Transformer with Augmented Rank and Improved Focus

Zhaolin Hu
Zhejiang University
Hangzhou, Zhejiang, China
12321165@zju.edu.cn

Hehe Fan*
Zhejiang University
Hangzhou, Zhejiang, China
hehefan@zju.edu.cn

Kun Li
United Arab Emirates University
Al Ain, United Arab Emirates
kunli.hfut@gmail.com

Yi Yang
Zhejiang University
Hangzhou, Zhejiang, China
yangyics@zju.edu.cn

Abstract

Linear attention mechanisms have emerged as efficient alternatives to full self-attention in Graph Transformers, offering linear time complexity. However, existing linear attention models often suffer from a significant drop in expressiveness due to low-rank projection structures and overly uniform attention distributions. We theoretically prove that these properties reduce the class separability of node representations, limiting the model's classification ability. To address this, we propose a novel hybrid framework that enhances both the rank and focus of attention. Specifically, we enhance linear attention by attaching a gated local graph network branch to the value matrix, thereby increasing the rank of the resulting attention map. Furthermore, to alleviate the excessive smoothing effect inherent in linear attention, we introduce a learnable log-power function into the attention scores to reduce entropy and sharpen focus. We theoretically show that this function decreases entropy in the attention distribution, enhancing the separability of learned embeddings. Extensive experiments on both homophilic and heterophilic graph benchmarks demonstrate that our method achieves competitive performance while preserving the scalability of linear attention.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → **Neural networks**.

Keywords

Graph, Graph Transformers, Node Classification

ACM Reference Format:

Zhaolin Hu, Kun Li, Hehe Fan, and Yi Yang. 2026. GraphTARIF: Linear Graph Transformer with Augmented Rank and Improved Focus. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792219>

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792219>

1 Introduction

Graph Neural Networks (GNNs) [16, 19, 21, 42, 48] have emerged as a powerful paradigm for analyzing graph-structured data in a variety of Web-related domains, including social networks [5], recommender systems [56], communication networks [27], and anomaly detection [9]. By leveraging message passing across graph neighborhoods, GNNs effectively capture local structural patterns and have achieved remarkable success in node classification, link prediction, and graph-level inference tasks. Nevertheless, their reliance on strictly local aggregation makes it inherently difficult to model long-range dependencies across distant nodes [1], thereby restricting expressiveness and limiting their ability to approximate complex relational functions in large-scale Web graphs [53].

Inspired by recent advances in Transformer-based models in language and vision domains [3, 8, 12, 23, 47], Graph Transformers (GTs) [33, 55] have gained increasing popularity in recent years. Unlike GNNs, GTs allow each node to directly attend to every other node in the graph, effectively creating a fully connected graph to mitigate locality bias and capture long-range interactions between distant nodes. The global self-attention (SA) mechanism can enhance the expressivity of the model and has demonstrated considerable success, especially in graph-level tasks such as molecular property prediction [33].

Nevertheless, the quadratic complexity of full self-attention ($O(N^2)$ for N nodes) poses a significant bottleneck for scaling Graph Transformers to node-level tasks on large-scale Web graphs, such as social networks or e-commerce interaction graphs, which often contain millions of nodes and edges. To address this challenge, recent works introduce linear self-attention (LSA) mechanisms [7, 50, 51], which approximate full softmax attention through kernel feature mappings or low-rank projections. This design reduces both memory and runtime costs from $O(N^2)$ to $O(N + |E|)$, enabling GTs to process much larger graphs more efficiently. Despite these computational benefits, empirical studies show that LSA-based Graph Transformers underperform even classical GNNs across a variety of benchmarks [32, 61], raising concerns about their effectiveness in Web applications where both scalability and accuracy are critical.

To study the limitations of linear GT and improve its performance, we first discover the loss of rank in overly uniform (high-entropy) attention distribution issues of LSA compared to full SA

through empirical studies. Intuitively, a low-rank LSA matrix restricts the model to express only a limited subset of distinct attention patterns, potentially collapsing the representations of different nodes or even different classes into similar embeddings. At the same time, high-entropy attention distributions lead to nearly uniform weights across nodes, preventing the model from sharply focusing on the most informative neighbors. To analyze the above two issues, we provide a theoretical analysis showing that the low-rank property of LSA matrices significantly reduces the differences between inter-class node representations, which has negative impact on node distinguishability. In addition, we conduct extensive experiments to demonstrate that high-entropy attention distributions hurt the performance of LSA by overly smoothing node representations.

To address these issues, we propose a novel linear Graph Transformer architecture (GraphTARIF) designed to increase the rank of attention scores and lower their entropy. Our method integrates a local module into linear attention to enhance expressiveness and introduces a learnable log-power function to sharpen attention distributions while maintaining training stability. The resulting model is a High-Rank Low-Entropy linear graph transformer, tailored explicitly for node-level tasks. We achieve competitive results in a wide range of homophilic and heterophilic graph learning tasks. Our contributions are summarized below:

- We theoretically prove that the low-rank and high-entropy properties of linear self-attention significantly reduce inter-class distinguishability, providing a principled explanation for the underperformance of linear Graph Transformers on node-level tasks.
- We introduce GraphTARIF, a linear Graph Transformer that integrates a local enhancement module and a learnable log-power function to simultaneously improve attention rank and reduce entropy, striking a balance between scalability and expressiveness.
- We evaluate GraphTARIF across a wide spectrum of Web-related datasets, *i.e.*, e-commerce, social networks, crowdsourcing platforms, and Wikipedia-based graphs, and it consistently outperforms existing baselines.

2 Related Work

Graph Neural Networks. Graph Neural Networks (GNNs) have emerged as powerful methods for node classification tasks, primarily relying on local neighborhood aggregation through message passing. For example, Graph Convolutional Networks (GCNs) [21] aggregate neighborhood information by leveraging a normalized adjacency matrix to propagate features. GraphSAGE [16] generalizes this approach by sampling and aggregating neighboring nodes, enabling inductive learning. Graph Attention Networks (GATs) [48] introduce attention mechanisms at the edge level, adaptively weighting neighbors based on their importance. Despite their success, these methods inherently suffer from local heterophily [28–30, 59, 60], oversmoothing [24, 31], and oversquashing [1, 46], limiting their capability to represent complex node interactions across long-range distances.

Graph Transformer. Graph Transformers (GTs) employ global self-attention mechanisms, which allow direct interactions between distant nodes, thus mitigating locality bias. Early graph transformer

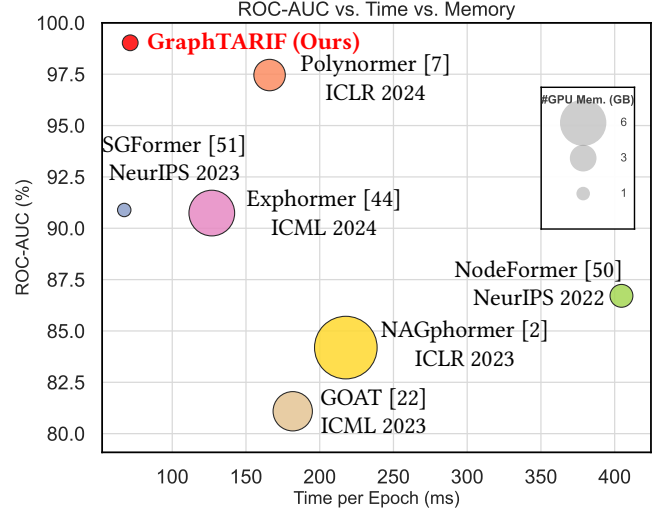


Figure 1: Comparisons among Graph Transformers(GT) in terms of accuracy, runtime, and GPU memory usage on Minesweeper. The proposed GraphTARIF achieves consistently superior accuracy with shorter runtime, demonstrating both effectiveness and efficiency.

models, such as Graph-BERT [57] and Graph Transformer [10], demonstrated the potential of self-attention for capturing long-range dependencies. However, the quadratic complexity of standard Transformer attention [47] is computationally prohibitive for large-scale graphs. Recent works have addressed this through linear attention approximations, such as Nodeformer [50] and SGformer [51], significantly improving computational efficiency. Nonetheless, studies have highlighted limitations of linear attention, specifically low-rank attention matrices [13] and uniformity (high-entropy) [17, 37] of attention score distributions, leading to suboptimal node classification performance. However, these problems are underexplored for linear graph transformers.

In this paper, we empirically demonstrate and theoretically analyze the negative impacts of these limitations. Then, we propose a novel Graph Transformer architecture to enhance attention rank and reduce attention entropy.

3 Preliminaries

Define a graph as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}, \mathbf{Y})$, where $\mathcal{V}$ is the set of nodes with $|\mathcal{V}| = n$, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes the set of edges. Let $\mathbf{X} \in \mathbb{R}^{n \times d}$ be the node feature matrix, where d is the feature dimension, and $\mathbf{Y} \in \mathbb{R}^{n \times C}$ the one-hot encoded label matrix with C classes. The adjacency matrix of $\mathcal{G}$ is denoted by $\mathbf{A} \in \mathbb{R}^{n \times n}$.

3.1 Graph Neural Networks

In the message passing framework [15], Graph Neural Networks (GNNs) update node representations by aggregating information from their neighbors. A typical GNN layer updates the representation of node i as:

$$h_i^{(l+1)} = f^{(l)} \left(h_i^{(l)}, \text{AGG}^{(l)} \left(\{h_j^{(l)} : j \in \mathcal{N}_i\} \right) \right), \quad (1)$$

where $h_i^{(l)}$ is the embedding of node i at the l -th layer, $\mathcal{N}_i$ denotes the neighbors of node i , $f^{(l)}$ is the activation function, and $\text{AGG}^{(l)}(\cdot)$ is an aggregation function such as sum, mean, or max pooling [53].

3.2 Graph Transformers

3.2.1 Standard Self-Attention. Transformers [47] use self-attention to capture global dependencies between all pairs of nodes. The attention mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (2)$$

$$Q = HW_Q, K = HW_K, V = HW_V, \quad (3)$$

where Q, K, V are query, key, and value matrices obtained via learned projections of input node features H , and d_k is the dimension of queries/keys. Although powerful, this formulation has $\mathcal{O}(N^2)$ time and space complexity, making it unsuitable for large-scale graphs.

3.2.2 Linear Attention. To address the challenge of scalability, linear attention mechanisms approximate softmax attention with a kernel function that allows rearranging the computation to linear complexity [20]:

$$\text{LinearAttention}(Q, K, V) = \text{Sim}(Q, K)V = \phi(Q)(\phi(K)^\top V), \quad (4)$$

where $\phi(\cdot)$ is a kernel feature map, such as positive random features or ReLU-based transformations [20]. This reduces the complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, enabling applications to large graphs. However, in the following section, we will show that such an approximation often leads to *low-rank* attention matrices and *high-entropy* attention distributions, which can limit expressiveness and harm model performance on node classification tasks.

3.3 Entropy

Entropy measures the uncertainty or dispersion of a sequence. Lower entropy indicates a more concentrated distribution, which often implies clearer semantic focus. In contrast, high entropy corresponds to smoother distributions that may dilute important signals. In this work, we adopt the definition of entropy for nonnegative sequences as proposed in prior work [37].

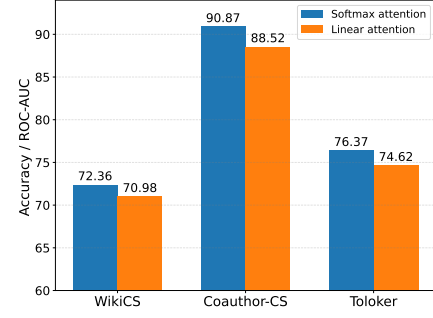
DEFINITION 1 (POSITIVE SEQUENCE ENTROPY (PSE) [37]). Let a sequence $\mathbf{x} = (x_1, \dots, x_N)$ with $x_i \geq 0$, and $s = \sum_{i=1}^N x_i > 0$. Then the entropy of this positive sequence is defined by:

$$\text{PSE}(\mathbf{x}) = - \sum_{i=1}^N \frac{x_i}{s} \log\left(\frac{x_i}{s}\right). \quad (5)$$

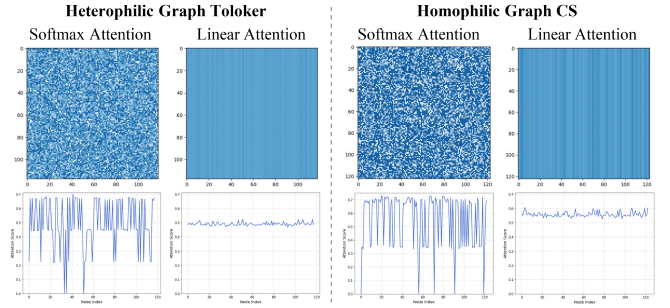
4 Motivation

In this section, we empirically and theoretically analyze the inherent limitations of linear attention in the context of node classification.

We build a minimal vanilla Graph Transformer architecture and evaluate it on three datasets: two homophilic graphs (WikiCS [38] and CS [43]) and one heterophilic graph (Toloker [39]). To incorporate structural information, we adopt Random Walk Structural Encoding [11] (RWSE) as the positional encoding method.



(a) Node classification accuracy



(b) Visualization of normalized attention matrices

Figure 2: (a) Node classification performance on homophilic graphs (WikiCS, CS) and a heterophilic graph (Toloker) shows that softmax attention consistently outperforms linear attention. (b) Visualization of normalized attention matrices for 120 evenly-sampled nodes reveals that linear attention produces low-rank, high-entropy attention distributions across both homophilic and heterophilic graphs.

As shown in Fig. 2 (a), across all three datasets, the variant with softmax attention significantly outperforms its linear counterpart. This trend holds true across both homophilic and heterophilic graphs. Similar to its behavior in textual and visual transformers, linear attention also underperforms compared to softmax attention in graph-based tasks.

To better understand the underlying reasons, we visualize the attention score matrices for both attention types. For clarity, we uniformly sample approximately 120 nodes from the graph and normalize the attention vectors row-wise for visualization. As shown in Fig. 2 (b), we observe that:

- Linear attention consistently produces **low-rank** attention matrices—much lower than the number of nodes N ;
- The attention weights are **smooth and uniform**, indicating high entropy compared to softmax attention.

These phenomena are observed in both homophilic and heterophilic graphs, suggesting that the issues are fundamental rather than dataset-specific.

To understand this behavior, let us examine the structural limitation of linear attention. Suppose $\phi(Q), \phi(K) \in \mathbb{R}^{N \times d}$. The attention score matrix is computed as:

$$M = \phi(Q)\phi(K)^\top \quad \text{with} \quad \text{rank}(M) \leq \min\{N, d\}. \quad (6)$$

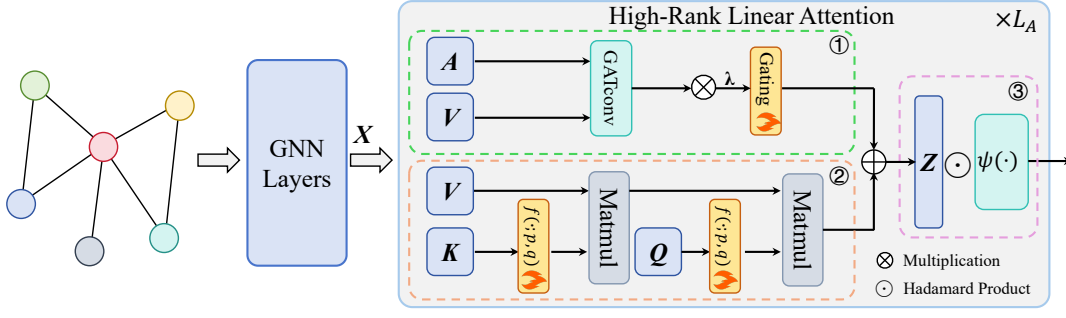


Figure 3: The overall framework of GraphTARIF. It mainly consists of GNN layers and high-rank linear attention. ① is the gated local attention that mitigates the low-rank limitation of linear attention. ② is a learnable log-power function $f(\cdot; p, q)$ that sharpens the attention distribution and reduces entropy, and ③ is a node-wise post-modulation module that produces clearer and more discriminative node representations. Here, Q, K, V denote the standard query, key, and value projections of node features X , A is the graph adjacency matrix, and $\psi(\cdot)$ denotes a simple linear projection applied for node-wise post-modulation.

Since d is typically small in practice, and N can range from tens to hundreds of thousands in large-scale graph datasets, the resulting attention matrix M suffers from a severe rank bottleneck. This limitation significantly restricts the model’s capacity to capture diverse and fine-grained interactions among nodes.

Furthermore, Zhang *et al.* [58] demonstrate that linear attention tends to produce smooth and high-entropy attention distributions. This issue becomes even more pronounced in graph-based tasks with a large number of nodes, where the attention weights are excessively dispersed. Such diffuse attention exacerbates the rank deficiency of the attention matrix, ultimately hindering the model’s expressiveness.

We now present two theoretical results that formally characterize how low-rank attention impacts the model’s ability to preserve class separability.

In node classification, the objective is to learn node representations such that nodes belonging to the same class are embedded closely together, while those from different classes are well-separated. A fundamental metric for evaluating this separability is the inter-class scatter matrix S_B , defined in its unweighted form as:

$$S_B := \frac{1}{K} \sum_{k=1}^K (\mu_k - \mu)(\mu_k - \mu)^\top, \quad (7)$$

where K is the number of classes, μ_k denotes the mean embedding of nodes in class k , and μ is the global mean of all node embeddings. The trace $\text{tr}(S_B)$ quantifies the total variance among class centroids. In this work, we adopt the trace of the inter-class scatter matrix, $\text{tr}(S_B)$, as a quantitative measure of inter-class variance in node classification tasks. Specifically, $\text{tr}(S_B)$ quantifies the total variance among class centroids. A larger $\text{tr}(S_B)$ indicates greater inter-class separation, which typically implies stronger discriminative power and higher classification accuracy.

THEOREM 1. Let $X \in \mathbb{R}^{n \times d}$ be the node feature matrix and $M \in \mathbb{R}^{n \times n}$ an attention matrix applied to transform the embeddings. Suppose that the rows of X are drawn from a Gaussian mixture model. The expected inter-class variance after applying the attention transformation, measured by the trace of the between-class scatter matrix,

is upper bounded by a term that scales linearly with the rank of M :

$$\mathbb{E}[\text{tr}(S_B(MX))] \leq C \cdot r, \quad (8)$$

where r is the rank of the attention matrix M , and C is a constant that depends on the underlying graph data and feature distribution.

The proof is provided in Appendix B.1. This theorem suggests that severely low-rank attention mechanisms may limit the ability to preserve class-discriminative information.

THEOREM 2. Let M_1 be an attention matrix, and let M_2 be obtained by applying a low-rank and smoothing transformation to M_1 , e.g., $M_2 = PM_1$, where $P \in \mathbb{R}^{n \times n}$ is a low-rank and smoothing operator. Then,

$$\mathbb{E}[\text{tr}(S_B(M_2X))] < \mathbb{E}[\text{tr}(S_B(M_1X))]. \quad (9)$$

The proof is given in Appendix B.2. This result formalizes the intuition that applying a smoother and lower-rank transformation to the attention matrix reduces its ability to preserve discriminative structure.

These results support our core hypothesis: the low-rank, overly smooth nature of linear attention limits its ability to preserve inter-class variation in node representations, explaining its degraded performance in node classification. Motivated by this, the remainder of the paper introduces architectural modifications designed to increase the effective rank of the attention matrix and sharpen attention distributions, thereby enhancing class separability and improving classification accuracy.

5 Methodology

Based on the observations and theoretical analysis presented in the previous section, we propose a novel Graph Transformer architecture specifically designed to address the low-rank and high-entropy limitations of linear attention. As illustrated in Fig. 3, our method comprises two core components: a High-Rank Linear Attention mechanism and a learnable log-power function aimed at reducing.

5.1 High-Rank Linear Attention

To address the inherent low-rank limitation of standard linear attention mechanisms, we propose a simple yet effective strategy to

enhance the rank of the attention output. In its conventional form, linear attention is computed as:

$$Z = \phi(Q) (\phi(K)^\top V). \quad (10)$$

Inspired by the previous works [17], we introduce an additional complementary branch to enrich the representation by injecting higher-rank local context. Specifically, we modify the attention output as follows:

$$Z = \phi(Q) (\phi(K)^\top V) + \text{GAT}(V), \quad (11)$$

where $\text{GAT}(V)$ denotes the local features generated by a Graph Attention Network (GAT). This modification is equivalent to implicitly augmenting the attention score matrix from $\phi(Q)\phi(K)^\top$ to $M_{\text{eq}} := \phi(Q)\phi(K)^\top + M_{\text{GAT}}$, in which M_{GAT} corresponds to the attention matrix produced by the GAT module, and M_{eq} refers to the resulting equivalent attention matrix. Since GAT assigns attention weights based on the local neighborhood defined by the graph structure, M_{GAT} is typically sparse, with each row normalized over the corresponding node’s immediate neighbors. Despite this sparsity, M_{GAT} can, in principle, attain a rank as high as $n - 1$. This greater rank capacity enables M_{eq} to capture more expressive and diverse node-level interactions, thus enhancing the model’s representational power.

However, we observe that simply incorporating GAT attention does not consistently improve performance [7]. We attribute this to the numerical dominance of GAT-derived attention scores in node classification tasks. Unlike vision transformers, where attention score matrices typically have only a few hundred dimensions, attention matrices in graph-based models often span tens of thousands of dimensions due to the large number of nodes. Moreover, since GAT assigns non-zero attention weights only to a node’s local neighbors, the resulting attention values tend to be significantly larger in magnitude than those from global linear attention. When combined without proper regulation, this disparity can lead to the suppression of global signals, thereby undermining the benefits of global context modeling.

To mitigate the dominance of local GAT attention in the hybrid formulation, we introduce a principled gating mechanism referred to as the Gating Mechanism that modulates its influence in a soft and learnable manner. Specifically, we incorporate a scalar gate into the GAT branch, resulting in the following enhanced formulation:

$$Z = \phi(Q) (\phi(K)^\top V) + \lambda \cdot \sigma(a) \cdot \text{GAT}(V), \quad (12)$$

where $\lambda > 0$ is a global scaling hyperparameter, and $\sigma(a)$ is a gating function (e.g., sigmoid) applied to a learnable scalar a . This design introduces a trainable regulator that balances global and local contributions during training.

Overall, the proposed hybrid attention mechanism retains the computational efficiency of linear attention while incorporating an interpretable and controllable local inductive bias. This formulation effectively increases the rank of the attention score matrix and enhances the model’s representational capacity. Moreover, based on our experiments and prior studies [52], global graph attention often allocates excessive weights to distant nodes, which leads to overly smooth output representations and weakens the preservation of node-specific information.

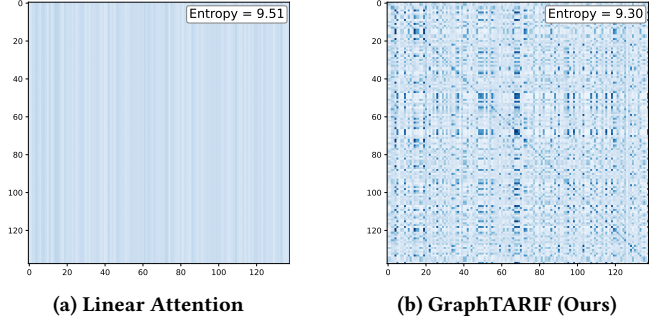


Figure 4: (a) Linear attention yields high-entropy attention maps with smooth, overly uniform outputs. (b) After applying the Learnable Log-Power Functions, the attention map becomes significantly sharper with lower entropy, allowing nodes to focus on more relevant regions and produce more diverse representations.

To alleviate this issue, we adopt a node-wise post-modulation strategy following the global attention propagation. Specifically, inspired by [13], we reweight the output of the attention mechanism using a transformed version of the original node features, denoted by $\psi(X)$. The complete form of our module is:

$$\tilde{Z} = \psi(X) \odot Z. \quad (13)$$

We use a simple linear projection to serve as the function $\psi(\cdot)$. This modulation adjusts each node’s output based on its own features, helping to retain node-specific information while incorporating contextual signals. Previous works [13] have shown that this operation can increase the rank of the output representations. We theoretically demonstrate that it also reduces the entropy of the output representations, resulting in sharper and more discriminative embeddings, as detailed in Appendix B.4.

In summary, we propose a simple yet effective architecture that increases the rank of the attention scores matrix and reduces the entropy of the output representations, thereby enhancing the model’s capacity to produce discriminative node embeddings. Detailed visualizations can be found in Figs. 8, and 9 of Appendix C.

5.2 Learnable Log-Power Sharpening Function

To mitigate the high-entropy problem commonly observed in standard linear attention mechanisms, we propose a novel attention modulation function that explicitly reduces the row-wise entropy of the attention distribution while maintaining training stability. According to the theoretical results established in [37], using a modulation function $f(x)$ that satisfies $f'(x) > 0$ and $f''(x) > 0$ to transform the queries and keys, e.g., setting $\bar{Q} = f(Q)$, $\bar{K} = f(K)$, can effectively reduce the Positive Sequence Entropy of the resulting attention matrix during attention computation.

Previous approaches [17, 37] have explored simple power functions such as $f(x) = x^p$ to sharpen attention distributions. While effective in compressing entropy in the small-value regime, these functions tend to grow rapidly for large x , leading to exploding gradients and instability during training.

Category	Model	Squirrel	Amazon-Ratings	Roman-Empire	Minesweeper	Questions	Tolokers	Avg. Rank
GNN	GCN [21]	38.67 ± 1.84	48.70 ± 0.63	73.69 ± 0.74	89.75 ± 0.52	76.09 ± 1.27	83.64 ± 0.67	11.2
	GAT [48]	35.62 ± 2.06	52.70 ± 0.62	88.75 ± 0.41	93.91 ± 0.35	76.79 ± 0.71	83.78 ± 0.43	8.5
	GraphSAGE [16]	36.09 ± 1.99	53.63 ± 0.39	85.74 ± 0.67	93.51 ± 0.57	76.44 ± 0.62	82.43 ± 0.44	8.8
	H2GCN [63]	35.10 ± 1.15	36.47 ± 0.20	60.11 ± 0.52	89.71 ± 0.31	63.59 ± 1.46	73.35 ± 1.01	16.3
	CPGNN [62]	30.04 ± 2.03	39.79 ± 0.77	63.96 ± 0.62	52.03 ± 5.46	65.96 ± 1.95	73.36 ± 1.01	16.1
	GPRGNN [6]	38.95 ± 1.99	44.88 ± 0.34	64.85 ± 0.27	86.24 ± 0.61	55.48 ± 0.91	72.94 ± 0.97	14.8
	FSGNN [35]	35.92 ± 1.32	52.74 ± 0.83	79.92 ± 0.56	90.08 ± 0.70	78.86 ± 0.92	82.76 ± 0.61	9.7
	GloGNN [25]	35.11 ± 1.24	36.89 ± 0.14	59.63 ± 0.69	51.08 ± 1.23	65.74 ± 1.19	73.39 ± 1.17	16.5
Tuned-GNN	tuned-GCN [32]	45.01 ± 1.63	53.80 ± 0.60	91.27 ± 0.20	97.86 ± 0.24	79.02 ± 0.60	–	3.0
	tuned-SAGE [32]	40.78 ± 1.40	55.40 ± 0.21	91.06 ± 0.27	97.77 ± 0.62	77.21 ± 1.28	–	4.8
	tuned-GAT [32]	41.73 ± 2.07	55.54 ± 0.51	90.63 ± 0.14	97.73 ± 0.73	77.95 ± 0.51	–	4.6
GT	GraphGPS [40]	39.67 ± 2.84	53.10 ± 0.42	82.00 ± 0.61	90.63 ± 0.67	71.73 ± 1.47	83.71 ± 0.48	9.5
	NAGphormer [2]	–	51.26 ± 0.72	74.34 ± 0.77	84.19 ± 0.66	68.17 ± 1.53	78.32 ± 0.95	13.2
	Expormer [44]	–	53.51 ± 0.46	89.03 ± 0.37	90.74 ± 0.53	73.94 ± 1.06	83.77 ± 0.78	8.2
	NodeFormer [50]	38.52 ± 1.57	43.86 ± 0.35	74.83 ± 0.81	87.71 ± 0.69	74.27 ± 1.46	78.10 ± 1.03	12.8
	SGFormer [51]	41.80 ± 2.27	48.01 ± 0.49	79.10 ± 0.32	90.89 ± 0.58	72.15 ± 1.31	–	10.4
	Polynormer [7]	40.87 ± 1.96	54.81 ± 0.49	92.55 ± 0.37	97.46 ± 0.36	78.92 ± 0.89	85.91 ± 0.74	3.8
	GQT [49]	42.72 ± 1.69	54.32 ± 0.41	90.98 ± 0.24	97.36 ± 0.35	78.94 ± 0.86	–	4.4
	GraphTARIF (Ours)	45.58 ± 1.91	55.86 ± 0.42	93.23 ± 0.38	99.03 ± 0.19	79.64 ± 0.71	86.60 ± 0.51	1.0

Table 1: Performance (Accuracy or ROC AUC % ± std) on heterophilic datasets with average rank (lower is better). We report ROC AUC for the Minesweeper, Tolokers, and Questions datasets, while Accuracy is reported for Roman-Empire, Amazon-Ratings, and Squirrel. We highlight the top **first, **second**, and **third** results per dataset.**

To overcome these limitations, we propose the following entropy-shaping function:

$$f(x; p, q) = x \cdot (\log(1 + x^p))^q, \quad (14)$$

where the exponents p and q are parameterized as $p = 1 + \alpha \cdot \sigma(w)$ and $q = 1 + \beta \cdot \sigma(w)$, with $\sigma(\cdot)$ denoting the sigmoid function, w learnable weights, and $\alpha, \beta > 0$ as hyperparameters. This function possesses several desirable properties for attention modulation, as formally stated in Theorem 3. The detailed proof is provided in Appendix B.3.

This function is applied element-wise to the transformed queries and keys before the attention map is computed. As illustrated in Fig. 4, it modulates the magnitude of each feature dimension to generate more concentrated and discriminative attention weights.

THEOREM 3. *Let $f(x) = x \cdot (\log(1 + x^p))^q$ with $p, q > 1$. Then:*

- (1) $f'(x) > 0$ and $f''(x) > 0$ for all $x > 0$, implying reduced attention entropy [37].
- (2) As $x \rightarrow \infty$, $f'(x) = O((\log x)^q)$, which grows slower than power functions (e.g., x^p), promoting smoother gradients and improved training stability.

Moreover, our function introduces learnable degrees of freedom through the parameters of both inner (p) and outer (q) transformations, enabling more fine-grained control over attention sharpness. This enables adaptive attention sharpening where needed, while preserving numerical stability.

6 Experiments

6.1 Experimental Setup

6.1.1 Datasets. We evaluate our model on five widely used homophilic graph datasets and six heterophilous graph datasets. The homophilic graphs include Computer, Photo [36], CS, Physics [43],

and WikiCS [38]. The heterophilic graphs in our study are Squirrel [41], Roman-Empire, Amazon-ratings, Minesweeper, Tolokers, and Questions [39]. For these datasets, we adopt the data splitting strategy proposed in [7, 32]. In addition, to examine the scalability of our approach, we evaluate it on three large-scale graphs: ogbn-arxiv, ogbn-products, and Pokec. These datasets contain node sizes ranging from 0.16M up to 2.4M, and we follow the standard evaluation settings of [7].

6.1.2 Baselines. We compare the proposed model against a broad set of baselines covering both classical GNNs and recent Graph Transformer architectures. Specifically, we include: (i) classical GNNs such as GCN, and GraphSAGE; (ii) heterophily-specialized GNNs, such as H2GCN, CPGNN, and TunedGNN, which is a recent framework that enhances traditional GNNs through skip-connections and hyperparameter optimization; and (iii) Graph Transformer models, such as GraphGPS, Expormer, and NodeFormer. These baselines span a wide range of model paradigms and design philosophies, and collectively represent the current state-of-the-art under both homophilic and heterophilic settings. For baseline methods, we report the results as documented in prior work. For our method, we report the average performance over five independent runs.

6.2 Main Comparison

We evaluate the proposed model, GraphTARIF, on a comprehensive set of node classification benchmarks, including both homophilic and heterophilic graph datasets. The results are summarized in Tab. 1 and Tab. 2, where we report the classification accuracy (%) or ROC-AUC score, along with the average rank across datasets. We compare against three major categories of baselines: GNNs, Graph Transformers (GT), and Tuned-GNN.

Category	Model	Computer	Photo	CS	Physics	WikiCS	Avg. Rank
GNN	GCN [21]	89.65 ± 0.52	92.70 ± 0.20	92.92 ± 0.12	96.18 ± 0.07	77.47 ± 0.85	17.2
	GAT [48]	90.78 ± 0.13	93.87 ± 0.11	93.61 ± 0.14	96.17 ± 0.08	76.91 ± 0.82	16.8
	GraphSAGE [16]	91.20 ± 0.29	94.59 ± 0.14	93.91 ± 0.13	96.49 ± 0.06	74.77 ± 0.95	14.4
	APNP [14]	90.18 ± 0.17	94.32 ± 0.14	94.49 ± 0.07	96.54 ± 0.07	78.87 ± 0.11	13.4
	GCNII [4]	91.04 ± 0.41	94.30 ± 0.20	92.22 ± 0.14	95.97 ± 0.11	78.68 ± 0.55	15.8
	GPRGNN [6]	89.32 ± 0.29	94.49 ± 0.14	95.13 ± 0.09	96.85 ± 0.08	78.12 ± 0.23	13.4
	GGCN [54]	91.81 ± 0.20	94.50 ± 0.11	95.25 ± 0.05	97.07 ± 0.05	78.44 ± 0.53	10.4
	OrderedGNN [45]	92.03 ± 0.13	95.10 ± 0.20	95.00 ± 0.10	97.00 ± 0.08	79.01 ± 0.68	8.8
	tGNN [18]	83.40 ± 1.33	89.92 ± 0.72	92.85 ± 0.48	96.24 ± 0.24	71.49 ± 1.05	19.2
Tuned-GNN	tuned-GCN [32]	93.99 ± 0.12	96.10 ± 0.46	96.17 ± 0.06	97.46 ± 0.10	80.30 ± 0.62	3.6
	tuned-SAGE [32]	93.25 ± 0.14	96.78 ± 0.23	96.38 ± 0.11	97.19 ± 0.05	80.69 ± 0.31	4.0
	tuned-GAT [32]	94.09 ± 0.37	96.60 ± 0.33	96.21 ± 0.14	97.25 ± 0.06	81.07 ± 0.54	3.0
GT	GraphGPS [40]	91.19 ± 0.54	95.06 ± 0.13	93.93 ± 0.12	97.12 ± 0.19	78.66 ± 0.49	11.4
	NAGphormer [2]	91.22 ± 0.14	95.49 ± 0.11	95.75 ± 0.09	97.34 ± 0.03	77.16 ± 0.72	8.6
	Expformer [44]	91.47 ± 0.17	95.35 ± 0.22	94.93 ± 0.01	96.89 ± 0.09	78.54 ± 0.49	10.4
	NodeFormer [50]	86.98 ± 0.62	93.46 ± 0.35	95.64 ± 0.22	96.45 ± 0.28	74.73 ± 0.94	15.6
	SGFormer [51]	91.99 ± 0.70	95.10 ± 0.47	94.78 ± 0.20	96.60 ± 0.18	73.46 ± 0.56	12.4
	Polynormer [7]	93.68 ± 0.21	96.46 ± 0.26	95.53 ± 0.16	97.27 ± 0.08	80.10 ± 0.67	5.4
	GQT [49]	93.37 ± 0.44	95.73 ± 0.18	96.11 ± 0.09	97.53 ± 0.06	80.14 ± 0.57	4.4
	GraphTARIF (Ours)	94.61 ± 0.17	97.03 ± 0.19	96.51 ± 0.11	97.39 ± 0.07	80.93 ± 0.57	1.6

Table 2: Performance (Accuracy % ± std) on homophilic datasets with average rank (lower is better).

6.2.1 Results on Heterophilic Datasets. As shown in Tab. 1, our model consistently achieves the best or comparable performance across all six heterophilic datasets: Squirrel, Amazon-Ratings, Roman-Empire, Minesweeper, Questions, and Tolokers. It ranks first on every dataset where evaluation is available, clearly outperforming both tuned GNN variants and state-of-the-art graph transformers. The overall average rank is 1.0, highlighting the robustness and effectiveness of our approach in handling heterophilic graph structures, where traditional GNNs and attention mechanisms often struggle to generalize.

6.2.2 Results on Homophilic Datasets. As shown in Tab. 2, our model achieves the best or comparable performance across all five homophilic datasets. Our method achieves the highest average accuracy on the Computer (94.61%), Photo (97.03%), and CS (96.51%) datasets, and obtains the best average rank of 1.4 among all 19 baseline models. Considering that these baselines—such as the Tuned-GNN [32] models—have been carefully optimized on these mature benchmarks, the improvements achieved by our model are noteworthy and demonstrate its significant potential. These results validate that our model not only maintains high expressiveness on classical heterophilic graphs but also generalizes well to classical homophilic graphs.

6.2.3 Results on Large-scale Graphs. To further demonstrate the scalability of our approach on large-scale graphs, we evaluate GraphTARIF on three widely used benchmarks: ogbn-arxiv, pokec, and ogbn-products. Among these benchmarks, ogbn-arxiv and ogbn-products are homophilic graphs, while pokec is heterophilic. As shown in Tab. 3, GraphTARIF achieves the best performance on ogbn-arxiv and on pokec. On ogbn-products, GraphTARIF also delivers highly competitive results. These results demonstrate that GraphTARIF can effectively scale to large-scale graphs and consistently achieves superior or competitive performance compared to strong baselines.

Model	ogbn-arxiv	pokec	ogbn-products
GCN [21]	71.74 ± 0.29	75.45 ± 0.17	75.64 ± 0.21
GAT [48]	72.01 ± 0.20	72.23 ± 0.18	79.45 ± 0.59
GPRGNN [6]	71.10 ± 0.12	78.83 ± 0.06	79.76 ± 0.59
LINKX [26]	66.18 ± 0.33	82.04 ± 0.07	71.59 ± 0.71
Tuned-GCN [32]	73.53 ± 0.12	86.33 ± 0.17	82.33 ± 0.19
Tuned-SAGE [32]	73.00 ± 0.28	85.97 ± 0.21	83.89 ± 0.36
Tuned-GAT [32]	73.30 ± 0.18	86.19 ± 0.23	80.99 ± 0.16
GraphGPS [40]	70.97 ± 0.41	OOM	OOM
NAGphormer [2]	70.13 ± 0.55	76.59 ± 0.25	73.55 ± 0.21
Expformer [44]	72.14 ± 0.28	OOM	OOM
NodeFormer [50]	67.19 ± 0.83	71.01 ± 0.20	72.93 ± 0.13
GOAT [22]	72.41 ± 0.40	66.37 ± 0.94	82.00 ± 0.43
SGFormer [51]	72.63 ± 0.13	73.76 ± 0.24	74.16 ± 0.31
Polynormer [7]	73.46 ± 0.16	86.10 ± 0.05	83.82 ± 0.11
GQT [49]	73.14 ± 0.16	83.76 ± 0.24	82.46 ± 0.17
GraphTARIF (Ours)	73.81 ± 0.11	86.55 ± 0.07	83.65 ± 0.27

Table 3: Performance (Accuracy % ± std) on large-scale datasets. OOM indicates out of memory.

6.3 Ablation Study

To assess the contribution of each component in GraphTARIF, we conduct an ablation study by progressively removing key modules, as summarized in Tab. 4. We evaluated four critical components: (1) the node-wise post-modulation $\psi(X) \odot Z$, which reweights the attention output based on transformed node features; (2) the sharpening function $f(\cdot)$, a learnable log-power transformation that reduces attention entropy; (3) the rank augmentation branch, implemented via a GAT module, increases the rank of the attention map.; and (4) the learnable GAT gate mechanism, which regulates the influence of the GAT branch through a gated scalar. We observe that removing any single component leads to a consistent performance drop across all datasets. In particular, removing both post-modulation and the sharpening function further exacerbates

Model Variant	$\psi(X) \odot Z$	$f(\cdot)$	Rank-Aug.	Gating Mechanism	Minesweeper	Roman-Empire	Computer	Photo
GraphTARIF (Full)	✓	✓	✓	✓	99.03 ± 0.19	93.23 ± 0.38	94.61 ± 0.17	97.03 ± 0.19
w/o Post-Modu.		✓	✓	✓	97.93 ± 0.44	90.76 ± 0.55	94.24 ± 0.16	96.69 ± 0.10
w/o Post-Modu. & Sharpening Func.			✓	✓	94.46 ± 0.89	90.44 ± 0.88	94.10 ± 0.35	96.34 ± 0.25
w/o Gating Mechanism	✓	✓	✓		98.05 ± 0.44	92.23 ± 0.34	94.23 ± 0.25	96.61 ± 0.27
Vanilla Linear attention					91.62 ± 0.54	89.39 ± 0.42	93.75 ± 0.48	96.30 ± 0.29

Table 4: Ablation study of GraphTARIF. “ $\psi(X) \odot Z$ ” denotes node-wise post-modulation, and “ $f(\cdot)$ ” denotes the sharpening function, “Rank-Aug.” represents the GAT-based rank augmentation branch.

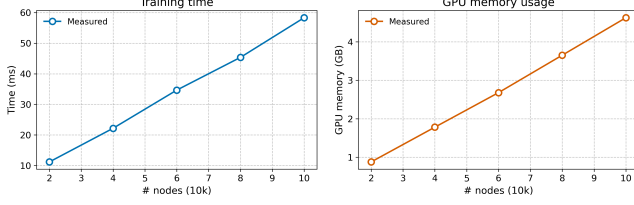


Figure 5: Training time and GPU memory usage of GraphTARIF on the pokec dataset.

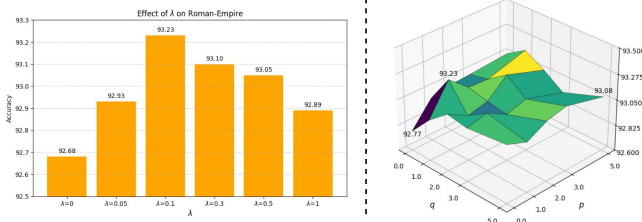


Figure 6: Impact of the hyperparameters λ (left), q , and p (right) on the Roman-Empire dataset.

the decline, confirming the importance of entropy reduction (third row). Additionally, removing the learnable GAT gate (fourth row) while keeping the GAT branch active also leads to a decline in performance, suggesting that unregulated local attention can interfere with global signal modeling. The full model, which combines all components, consistently achieves the best results across all datasets. These results collectively validate that each component in GraphTARIF plays a complementary and non-redundant role.

6.4 Additional Experiments

6.4.1 Model Scalability. This experiment aims to comprehensively evaluate the scalability of GraphTARIF. Specifically, we randomly sampled nodes from the training set, incrementally increasing the sample size from 20,000 to 100,000, and monitored the corresponding training time and GPU memory consumption as the dataset size changed. Fig. 5 (left) and (right) illustrate the variation in runtime and GPU memory usage, respectively, on the pokec dataset. Evidently, both the runtime and memory consumption of GraphTARIF increase linearly with the growth in graph size, indicating that GraphTARIF exhibits linear time and space complexity. These results clearly demonstrate the scalability of GraphTARIF.

6.4.2 Hyperparameter Analysis. These experiments aim to intuitively illustrate the effects of hyperparameter selection. Specifically, we test three key hyperparameters of our method (λ , q , and p) on the Roman-Empire dataset. As shown in Fig. 6, the left panel presents the influence of the coefficient λ . Evidently, a moderate value ($\lambda = 0.1$) yields the highest accuracy. The right panel of Fig. 6 illustrates the impact of the hyperparameters p and q in the Learnable Log-Power Functions on the Roman-Empire dataset. We observe that the model performance peaks when p and q are set around 1–2. Beyond this range, the performance gradually stabilizes, indicating that moderate values of p and q are sufficient to enhance the model’s expressiveness. Through automated hyperparameter optimization, we further determined that the optimal values of p and q across different datasets typically lie within the range of 1 to 3. At the same time, the model maintains competitive performance even with larger values, further validating the stability of our approach.

6.4.3 Efficiency Comparison. As shown in Fig. 1, compared with recent Graph Transformers, GraphTARIF achieves superior performance while requiring less runtime and memory. It attains the best results, and it incurs lower computational costs than Polynormer and Exphormer, demonstrating its effectiveness and efficiency. Moreover, our method achieves strong performance with a relatively small number of parameters, as detailed in Table 6 of Appendix C.

7 Conclusion

In this paper, we introduced GraphTARIF, a hybrid linear Graph Transformer that augments the rank of attention matrices and reduces entropy through a gated local branch and a learnable log-power function. Theoretical analysis and extensive experiments demonstrate that GraphTARIF achieves competitive expressiveness while preserving linear scalability, consistently outperforming strong baselines on both homophilic and heterophilic benchmarks as well as large-scale graphs.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (92570101) and the Fundamental Research Funds for the Central Universities (226-202500080).

References

- [1] Uri Alon and Eran Yahav. 2020. On the Bottleneck of Graph Neural Networks and its Practical Implications. In *International Conference on Learning Representations*.
- [2] Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. 2023. NAGphormer: A tokenized graph transformer for node classification in large graphs. In *The Eleventh International Conference on Learning Representations*.

- [3] Kerui Chen, Zhihang Wu, Wenjin Hou, Kun Li, Hehe Fan, and Yi Yang. 2025. Prompt-aware controllable shadow removal. *arXiv preprint arXiv:2501.15043* (2025).
- [4] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *International conference on machine learning*. 1725–1735.
- [5] Zhengdao Chen, Xiang Li, and Joan Bruna. 2017. Supervised community detection with line graph neural networks. *arXiv preprint arXiv:1705.08415* (2017).
- [6] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988* (2020).
- [7] Chenhui Deng, Zichao Yue, and Zhiru Zhang. 2024. Polynormer: Polynomial-Expressive Graph Transformer in Linear Time. In *The Twelfth International Conference on Learning Representations*.
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.
- [9] Yingdong Dou, Zhiwei Liu, Li Sun, Yutong Deng, Hao Peng, and Philip S Yu. 2020. Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 315–324.
- [10] Vijay Prakash Dwivedi and Xavier Bresson. 2020. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699* (2020).
- [11] Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2021. Graph neural networks with learnable structural and positional representations. *arXiv preprint arXiv:2110.07875* (2021).
- [12] Hehe Fan, Yi Yang, and Mohan Kankanhalli. 2022. Point spatio-temporal transformer networks for point cloud video modeling. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 2 (2022), 2181–2192.
- [13] Qihang Fan, Huaibo Huang, and Ran He. 2024. Breaking the Low-Rank Dilemma of Linear Attention. *arXiv preprint arXiv:2411.07635* (2024).
- [14] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv preprint arXiv:1810.05997* (2018).
- [15] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. 1263–1272.
- [16] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems* 30 (2017).
- [17] Dongchen Han, Xuran Pan, Yizeng Han, Shiji Song, and Gao Huang. 2023. Flatten transformer: Vision transformer using focused linear attention. In *Proceedings of the IEEE/CVF international conference on computer vision*. 5961–5971.
- [18] Chenqing Hua, Guillaume Rabusseau, and Jian Tang. 2022. High-order pooling for graph neural networks with tensor decomposition. *Advances in Neural Information Processing Systems* 35 (2022), 6021–6033.
- [19] Yingzhao Jian, Yue Zhang, Ying Wei, Hehe Fan, and Yi Yang. 2025. Reaction Graph: Towards Reaction-Level Modeling for Chemical Reactions with 3D Structures. In *Forty-second International Conference on Machine Learning*.
- [20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*. PMLR, 5156–5165.
- [21] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [22] Kezhi Kong, Jiuhai Chen, John Kirchenbauer, Renkun Ni, C Bayan Bruss, and Tom Goldstein. 2023. GOAT: A global transformer on large-scale graphs. In *International Conference on Machine Learning*. 17375–17390.
- [23] Kun Li, Dan Guo, and Meng Wang. 2023. ViGT: proposal-free video grounding with a learnable token in the transformer. *Science China Information Sciences* 66, 10 (2023), 202102.
- [24] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [25] Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. 2022. Finding global homophily in graph neural networks when meeting heterophily. In *International conference on machine learning*. PMLR, 13242–13256.
- [26] Derek Lim, Felix Hohne, Xiuyu Li, Sijia Linda Huang, Vaishnavi Gupta, Omkar Bhalerao, and Ser Nam Lim. 2021. Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods. *Advances in Neural Information Processing Systems* 34 (2021), 20887–20902.
- [27] Qincheng Lu, Sitao Luan, and Xiao-Wen Chang. 2024. GCEPNet: Graph Convolution-Enhanced Expectation Propagation for Massive MIMO Detection. *arXiv preprint arXiv:2404.14886* (2024).
- [28] Qincheng Lu, Jiaqi Zhu, Sitao Luan, and Xiao-Wen Chang. 2024. Flexible diffusion scopes with parameterized laplacian for heterophilic graph learning. *arXiv preprint arXiv:2409.09888* (2024).
- [29] Sitao Luan, Chenqing Hua, Qincheng Lu, Liheng Ma, Lirong Wu, Xinyu Wang, Minkai Xu, Xiao-Wen Chang, Doina Precup, Rex Ying, et al. 2024. The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges. *arXiv preprint arXiv:2407.09618* (2024).
- [30] Sitao Luan, Chenqing Hua, Minkai Xu, Qincheng Lu, Jiaqi Zhu, Xiao-Wen Chang, Jie Fu, Jure Leskovec, and Doina Precup. 2024. When Do Graph Neural Networks Help with Node Classification? Investigating the Homophily Principle on Node Distinguishability. *Advances in Neural Information Processing Systems* 36 (2024).
- [31] Sitao Luan, Mingde Zhao, Xiao-Wen Chang, and Doina Precup. 2019. Break the ceiling: Stronger multi-scale deep graph convolutional networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [32] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2024. Classic gnns are strong baselines: Reassessing gnns for node classification. *Advances in Neural Information Processing Systems* 37 (2024), 97650–97669.
- [33] Liheng Ma, Chen Lin, Derek Lim, Adriana Romero-Soriano, Puneet K Dokania, Mark Coates, Philip Torr, and Ser-Nam Lim. 2023. Graph inductive biases in transformers without message passing. In *International Conference on Machine Learning*. 23321–23337.
- [34] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.
- [35] Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2022. Simplifying approach to node classification in Graph Neural Networks. *Journal of Computational Science* 62 (2022), 101695.
- [36] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
- [37] Weikang Meng, Yadan Luo, Xin Li, Dongmei Jiang, and Zheng Zhang. 2025. PolaFormer: Polarity-aware linear attention for vision transformers. *arXiv preprint arXiv:2501.15061* (2025).
- [38] Péter Mernyei and Cătălina Cangea. 2020. Wiki-cs: A wikipedia-based benchmark for graph neural networks. *arXiv preprint arXiv:2007.02901* (2020).
- [39] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? *arXiv preprint arXiv:2302.11640* (2023).
- [40] Ladislav Rampásek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. Recipe for a general, powerful, scalable graph transformer. *Advances in Neural Information Processing Systems* 35 (2022), 14501–14515.
- [41] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. 2021. Multi-scale attributed node embedding. *Journal of Complex Networks* 9, 2 (2021), cnab014.
- [42] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE Transactions on Neural Networks* 20, 1 (2008), 61–80.
- [43] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [44] Hamed Shirzad, Ameya Velingker, Balaji Venkatachalam, Danica J Sutherland, and Ali Kemal Sinop. 2023. Expformer: Sparse transformers for graphs. In *International Conference on Machine Learning*. 31613–31632.
- [45] Yunchong Song, Chenghu Zhou, Xinbing Wang, and Zhouhan Lin. 2023. Ordered gnn: Ordering message passing to deal with heterophily and over-smoothing. *arXiv preprint arXiv:2302.01524* (2023).
- [46] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. 2021. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522* (2021).
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [49] Limei Wang, Kaveh Hassani, Si Zhang, Dongqi Fu, Baichuan Yuan, Weilin Cong, Zhigang Hua, Hao Wu, Ning Yao, and Bo Long. 2025. Learning Graph Quantized Tokenizers. In *The Thirteenth International Conference on Learning Representations*.
- [50] Qitian Wu, Wentao Zhao, Zenan Li, David P Wipf, and Junchi Yan. 2022. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems* 35 (2022), 27387–27401.
- [51] Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. 2023. Sgformer: Simplifying and empowering transformers for large-graph representations. *Advances in Neural Information Processing Systems* 36 (2023), 64753–64773.
- [52] Yujie Xing, Xiao Wang, Yibo Li, Hai Huang, and Chuan Shi. 2024. Less is more: on the over-globalizing problem in graph transformers. *arXiv preprint arXiv:2405.01102* (2024).

- [53] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).
- [54] Yujun Yan, Milad Hashemi, Kevin Swersky, Yaoqing Yang, and Danai Koutra. 2022. Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *2022 IEEE International Conference on Data Mining*. IEEE, 1287–1292.
- [55] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in Neural Information Processing Systems* 34 (2021), 28877–28888.
- [56] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
- [57] Jiawei Zhang, Haopeng Zhang, Congying Xia, and Li Sun. 2020. Graph-bert: Only attention is needed for learning graph representations. *arXiv preprint arXiv:2001.05140* (2020).
- [58] Michael Zhang, Kush Bhatia, Hermann Kumbong, and Christopher Ré. 2024. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. *arXiv preprint arXiv:2402.04347* (2024).
- [59] Yilun Zheng, Xiang Li, Sitao Luan, Xiaojiang Peng, and Lihui Chen. 2025. Let your features tell the differences: Understanding graph convolution by feature splitting. In *The Thirteenth International Conference on Learning Representations*.
- [60] Yilun Zheng, Sitao Luan, and Lihui Chen. 2024. What is missing for graph homophily? disentangling graph homophily for graph neural networks. *Advances in Neural Information Processing Systems* 37 (2024), 68406–68452.
- [61] Jiajun Zhou, Xuanze Chen, Chenxuan Xie, Yu Shanqing, Qi Xuan, and Xiaoniu Yang. 2024. Rethinking Graph Transformer Architecture Design for Node Classification. *arXiv preprint arXiv:2410.11189* (2024).
- [62] Jiong Zhu, Ryan A Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K Ahmed, and Danai Koutra. 2021. Graph neural networks with heterophily. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 11168–11176.
- [63] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.

A Datasets and Experimental Details

Table 5 summarizes the statistics of all thirteen datasets used in our experiments, along with the corresponding major hyperparameter settings. We adopt GATConv as the base GNN operator preceding the attention layer, combined with residual connections, layer normalization, and ReLU activation. The attention module employs a linear attention mechanism with a sigmoid kernel feature map, enabling efficient global information aggregation. After the attention layer, one or two additional GNN layers are stacked to refine node representations and perform classification. All models are trained in a semi-supervised framework using the PyTorch Geometric (PyG) library. Experiments are conducted on a GPU cluster equipped with NVIDIA RTX 4090, RTX A6000, and L40S devices. Hyperparameters are optimized using Bayesian optimization via Optuna toolkit.

B Theoretical Proof

B.1 Proof for Theorem 1

We make the following assumptions and define the symbols used in the theorem:

- **Attention Matrix:** Let $M \in \mathbb{R}^{n \times n}$ be the attention matrix, where the elements $M_{ij} \in [0, 1]$ for all i, j , and each row of the matrix is normalized, e.g., for all i ,

$$\sum_{j=1}^n M_{ij} = 1. \quad (15)$$

- **Node Feature Matrix:** Let $X \in \mathbb{R}^{n \times d}$ be the node feature matrix, where n is the number of nodes and d is the dimensionality of the node features. There are K classes, and the

i -th row of X is denoted by $\mathbf{x}_i$. The set of nodes in class k is denoted as C_k , and its size is $n_k = |C_k|$. The node labels are $y_i \in \{1, 2, \dots, K\}$, and the features of each node are drawn from a class-conditional Gaussian distribution:

$$\mathbf{x}_i \sim \mathcal{N}(\boldsymbol{\mu}_{y_i}, \Sigma), \quad (16)$$

where $\boldsymbol{\mu}_{y_i}$ is the mean of class y_i , and Σ is the shared covariance matrix for all classes.

- **Class Centers:** The class centers are assumed to be orthogonal to each other, e.g., for any $i \neq j$,

$$\boldsymbol{\mu}_i^\top \boldsymbol{\mu}_j = 0. \quad (17)$$

- **Covariance Matrix:** The covariance matrix is assumed to be $\Sigma = \sigma^2 I$, where I is the identity matrix and σ^2 is a constant scalar value.

With the above assumptions, we now state the complete form of Theorem 1:

Theorem 1: Let $M \in \mathbb{R}^{n \times n}$ be the attention matrix, and $X \in \mathbb{R}^{n \times d}$ be the node feature matrix. Under the assumption that the attention matrix M is normalized and data-independent, and the node features X follow a class-conditional Gaussian distribution, we have the following relationship between the rank of the attention matrix and the expected value of the trace of the between-class scatter matrix:

$$\mathbb{E}[\text{tr}(S_B)] \leq C \cdot r, \quad (18)$$

where $r = \text{rank}(M)$ is the rank of the attention matrix, and C is a constant.

PROOF. We first consider the feature transformation $X' = MX$. Each transformed feature $\mathbf{x}'_i = \sum_j M_{ij} \mathbf{x}_j$ is still Gaussian since $\mathbf{x}_j \sim \mathcal{N}(\boldsymbol{\mu}_{y_j}, \Sigma)$. For class k , the empirical center after transformation can be written as

$$\boldsymbol{\mu}'_k = (\boldsymbol{\alpha}^{(k)})^\top X, \quad \text{with } \alpha_j^{(k)} = \frac{1}{n_k} \sum_{i \in C_k} M_{ij},$$

and the global mean as $\boldsymbol{\mu}' = \boldsymbol{\gamma}^\top X$, where $\gamma_j = \frac{1}{n} \sum_{i=1}^n M_{ij}$.

The between-class scatter becomes

$$S_B = \frac{1}{K} \sum_{k=1}^K (\boldsymbol{\mu}'_k - \boldsymbol{\mu}')(\boldsymbol{\mu}'_k - \boldsymbol{\mu}')^\top = X^\top A X,$$

where

$$A = \frac{1}{K} \sum_{k=1}^K (\boldsymbol{\alpha}^{(k)} - \boldsymbol{\gamma})(\boldsymbol{\alpha}^{(k)} - \boldsymbol{\gamma})^\top.$$

Hence

$$\mathbb{E}[\text{tr}(S_B)] = \text{tr}(A \mathbb{E}[XX^\top]).$$

We decompose $X = Y + E$, with Y the deterministic class means and E Gaussian noise. This gives

$$\mathbb{E}[XX^\top] = YY^\top + d\sigma^2 I_n,$$

so that

$$\mathbb{E}[\text{tr}(S_B)] = \text{tr}(A YY^\top) + d\sigma^2 \text{tr}(A).$$

Two auxiliary bounds are needed:

Lemma 1. $\|\boldsymbol{\alpha}^{(k)} - \boldsymbol{\gamma}\| \leq \sqrt{n}$, since both are convex combinations of $\{M_{ij}\}$.

Lemma 2. From Lemma 1,

$$\|A\|_F \leq \frac{n}{\sqrt{K}}, \quad \lambda_{\max}(A) \leq \|A\|_F \leq \frac{n}{\sqrt{K}}.$$

Dataset	Type	# nodes	# edges	# Features	Classes	Metric	GNN Layers	Attention Layers (L_A)	λ	p	q
Computer	Homophily	13,752	245,861	767	10	Accuracy	5	1	0.1	2.0	1.0
Photo	Homophily	7,650	119,081	745	8	Accuracy	8	1	0.1	2.0	2.0
CS	Homophily	18,333	81,894	6,805	15	Accuracy	5	3	0.1	1.0	2.0
Physics	Homophily	34,493	247,962	8,415	5	Accuracy	5	3	0.1	3.0	2.0
WikiCS	Homophily	11,701	216,123	300	10	Accuracy	5	2	0.1	1.0	2.0
Squirrel	Heterophily	2,223	46,998	2,089	5	Accuracy	4	1	0.1	2.0	1.0
tolokers	Heterophily	11,758	519,000	10	2	ROC-AUC	10	2	0.1	3.0	2.0
Roman-Empire	Heterophily	22,662	32,927	300	18	Accuracy	10	3	0.1	1.0	1.0
Amazon-Ratings	Heterophily	24,492	493,005	300	5	Accuracy	5	1	0.1	1.0	1.0
Minesweeper	Heterophily	10,000	39,304	7	5	ROC-AUC	10	2	0.1	2.0	1.5
Questions	Heterophily	48,921	153,540	301	2	ROC-AUC	8	1	0.3	2.0	1.0
ogbn-arxiv	Homophily (Large graphs)	169,343	1,166,243	128	40	Accuracy	7	1	0.2	3.0	0.1
ogbn-products	Homophily (Large graphs)	2,449,029	61,859,140	100	47	Accuracy	7	2	0.1	2.0	1.0
pokec	Heterophily (Large graphs)	1,632,803	30,622,564	65	2	Accuracy	7	2	0.1	2.0	1.0

Table 5: Overview of the datasets used for node classification.

Thus $\text{tr}(\mathbf{A}) \leq r \lambda_{\max}(\mathbf{A}) \leq \frac{nr}{\sqrt{K}}$, with $r = \text{rank}(\mathbf{A}) \leq \text{rank}(\mathbf{M})$.

Finally, since both $\mathbf{Y}\mathbf{Y}^\top$ and $\mathbf{A}$ are PSD,

$$\text{tr}(\mathbf{A}\mathbf{Y}\mathbf{Y}^\top) \leq \lambda_{\max}(\mathbf{Y}\mathbf{Y}^\top) \text{tr}(\mathbf{A}).$$

Combining the two terms yields

$$\mathbb{E}[\text{tr}(\mathbf{S}_B)] \leq (\lambda_{\max}(\mathbf{Y}\mathbf{Y}^\top) + d\sigma^2) \cdot \frac{nr}{\sqrt{K}}.$$

Let $C = \frac{n}{\sqrt{K}}(\lambda_{\max}(\mathbf{Y}\mathbf{Y}^\top) + d\sigma^2)$, independent of $\mathbf{M}$, we conclude

$$\mathbb{E}[\text{tr}(\mathbf{S}_B)] \leq C \cdot \text{rank}(\mathbf{M}).$$

□

Although Theorem 1 provides an upper bound on the inter-group variance, it still offers valuable insight in practice. Real-world data often exhibit complex, multi-modal, or manifold-structured distributions rather than simple class-wise Gaussian clusters. In such cases, a higher-rank attention matrix is still necessary to retain discriminative features across these intricate structures.

B.2 Proof for Theorem 2

Notation. We write $\mathbf{A} \preceq \mathbf{B}$ to indicate that $\mathbf{B} - \mathbf{A}$ is positive semidefinite (PSD), that is, for all $\mathbf{v} \in \mathbb{R}^n$,

$$\mathbf{v}^\top (\mathbf{B} - \mathbf{A}) \mathbf{v} \geq 0. \quad (19)$$

Equivalently, this means all eigenvalues of $\mathbf{B} - \mathbf{A}$ are nonnegative. When $\mathbf{A} \prec \mathbf{B}$, the difference is positive definite (PD).

Theorem 2: Let $\mathbf{M}_1 = \mathbf{P}\mathbf{M}_2$ with $\mathbf{P} \in \mathbb{R}^{n \times n}$. Since $\mathbf{P}$ is a smoothing operator, it is reasonable to assume that $\mathbf{P}$ satisfies $\mathbf{P}^\top \mathbf{P} \preceq \mathbf{I}$. Moreover, assume that $\text{rank}(\mathbf{M}_1) < \text{rank}(\mathbf{M}_2)$. Let the input feature matrix be $\mathbf{X} = \mathbf{Y} + \mathbf{E}$, where:

- $\mathbf{Y} \in \mathbb{R}^{n \times d}$ is the matrix of class centers with $\mathbf{Y}\mathbf{Y}^\top \succ 0$;
- $\mathbf{E} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ is isotropic Gaussian noise.

Then the expected inter-class variance after applying $\mathbf{M}_1$ is strictly smaller than that of $\mathbf{M}_2$:

$$\mathbb{E}[\text{tr}(\mathbf{S}_B(\mathbf{M}_1))] < \mathbb{E}[\text{tr}(\mathbf{S}_B(\mathbf{M}_2))]. \quad (20)$$

PROOF. For any attention matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, define

$$\boldsymbol{\alpha}^{(k)} = \frac{1}{n_k} \sum_{i \in C_k} \mathbf{M}_{i,:}, \quad \boldsymbol{\gamma} = \frac{1}{n} \sum_{i=1}^n \mathbf{M}_{i,:},$$

and

$$\mathbf{A} = \frac{1}{K} \sum_{k=1}^K (\boldsymbol{\alpha}^{(k)} - \boldsymbol{\gamma})(\boldsymbol{\alpha}^{(k)} - \boldsymbol{\gamma})^\top.$$

Under $\mathbf{M}$, the between-class scatter is

$$\mathbf{S}_B = \mathbf{X}^\top \mathbf{A} \mathbf{X}, \quad \mathbb{E}[\text{tr}(\mathbf{S}_B)] = \text{tr}(\mathbf{A}(\mathbf{Y}\mathbf{Y}^\top + d\sigma^2 \mathbf{I})).$$

Now consider $\mathbf{M}_1 = \mathbf{P}\mathbf{M}_2$, inducing $\mathbf{A}_1 = \mathbf{P}\mathbf{A}_2\mathbf{P}^\top$. Then

$$\mathbb{E}[\text{tr}(\mathbf{S}_B(\mathbf{M}_1))] = \text{tr}(\mathbf{P}\mathbf{A}_2\mathbf{P}^\top \mathbf{Y}\mathbf{Y}^\top) + d\sigma^2 \text{tr}(\mathbf{P}\mathbf{A}_2\mathbf{P}^\top).$$

Let $\mathbf{Q} = \mathbf{P}^\top \mathbf{P} \preceq \mathbf{I}$. For the signal term,

$$\text{tr}(\mathbf{P}\mathbf{A}_2\mathbf{P}^\top \mathbf{Y}\mathbf{Y}^\top) = \text{tr}(\mathbf{A}_2\mathbf{P}^\top \mathbf{Y}\mathbf{Y}^\top \mathbf{P}) < \text{tr}(\mathbf{A}_2\mathbf{Y}\mathbf{Y}^\top),$$

since $\mathbf{Y}\mathbf{Y}^\top \succ 0$ and $\mathbf{Q} \prec \mathbf{I}$. For the noise term,

$$\text{tr}(\mathbf{P}\mathbf{A}_2\mathbf{P}^\top) = \text{tr}(\mathbf{A}_2\mathbf{Q}) < \text{tr}(\mathbf{A}_2).$$

Thus both terms under $\mathbf{M}_1$ are strictly smaller than under $\mathbf{M}_2$, giving

$$\mathbb{E}[\text{tr}(\mathbf{S}_B(\mathbf{M}_1))] < \mathbb{E}[\text{tr}(\mathbf{S}_B(\mathbf{M}_2))].$$

□

B.3 Proof for Theorem 3

Theorem 3: Let $f(x) = x \cdot (\log(1+x^p))^q$, where $p > 1$, $q > 1$. Then:

- (1) $f'(x) > 0$ and $f''(x) > 0$ for all $x > 0$;
- (2) As $x \rightarrow \infty$, it holds that

$$f'(x) = \mathcal{O}((\log x)^q). \quad (21)$$

PROOF. We prove two claims.

(1) *Convexity.* Let $f(x) = x(\log(1+x^p))^q$ with $L(x) = \log(1+x^p)$. Then

$$f'(x) = L(x)^q + xqL(x)^{q-1} \frac{px^{p-1}}{1+x^p} > 0 \quad (x > 0).$$

Differentiating again and writing $f''(x) = A'(x) + B'(x)$ with $A(x) = L(x)^q$, $B(x) = xqL(x)^{q-1}L'(x)$, one obtains

$$B'(x) = pq \left[L(x)^{q-1}(L'(x) + xL''(x)) + (q-1)xL(x)^{q-2}(L'(x))^2 \right].$$

A direct calculation shows $L'(x) + xL''(x) = \frac{p^2 x^{p-1}}{(1+x^p)^2} > 0$. Since every term is nonnegative, we have $B'(x) > 0$ and $A'(x) > 0$, hence

$$f''(x) > 0 \quad \text{for all } x > 0.$$

Model	Roman-Empire		Questions	
	ROC-AUC	Params	Accuracy	Params
Expformer [44]	88.42±0.41	15.9M	73.94±1.06	10.7M
Polynormer [7]	92.55±0.37	9.91M	78.92±0.89	6.74M
GraphTARIF (Ours)	93.23±0.38	4.07M	79.64±0.71	5.17M

Table 6: ROC-AUC or Accuracy (%) and parameter counts (M) on two heterophilic graph datasets.

(2) *Asymptotic growth.* As $x \rightarrow \infty$, $\log(1 + x^p) \sim p \log x$, and

$$\frac{px^{p-1}}{1+x^p} \sim \frac{p}{x}, \quad x \cdot \frac{p}{x} = p.$$

Thus

$$f'(x) \sim (p \log x)^q + qp(p \log x)^{q-1} = O((\log x)^q).$$

Combining (1) and (2) proves the claim. $\square$

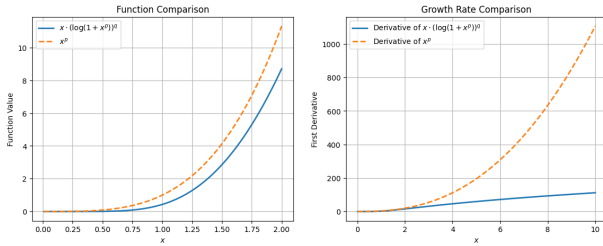


Figure 7: Comparison between $x \cdot (\log(1 + x^p))^q$ and x^p . Left: function values; Right: growth rates.

The proposed low-power function $f(x; p, q) = x \cdot (\log(1 + x^p))^q$, similar to the power function, amplifies differences in input values and thus reduces the entropy of the attention distribution, as shown in Fig. 7 (left). However, as illustrated in Fig. 7 (right), its derivative grows significantly more slowly when x is large. This property effectively mitigates the risk of exploding gradients during backpropagation and ensures more stable training.

B.4 Theoretical Analysis of Node-wise Post-modulation Effects on Output Features

In our framework, the node-wise post-modulation operation (Eq. 13) performs an element-wise transformation on the attention outputs. Prior work [13] demonstrates that such post-modulation can increase the rank of the feature matrix, thereby enhancing its representational capacity. Complementing this view, we theoretically show that post-modulation also *reduces the entropy* of node features—sharpening their distribution while preserving diversity, which ultimately benefits downstream classification.

Theorem 4. Let $X \in \mathbb{R}^{n \times d}$ denote node features, and $M \in \mathbb{R}^{n \times n}$ be an attention matrix emphasizing same-class nodes. Define

$$Y_1 = MX, \quad Y_2 = (MX) \odot X. \quad (22)$$

Then for each feature dimension k ,

$$\text{PSE}(Y_2^{(:,k)}) < \text{PSE}(Y_1^{(:,k)}), \quad (23)$$

where $\text{PSE}(\cdot)$ denotes the Positive Sequence Entropy.

PROOF. We analyze one feature dimension $\mathbf{x} = (x_1, \dots, x_n)$. For two nodes $x = (a, b)$ with $a > b > 0$, define $c = a/b > 1$. The normalized entropy is

$$\text{PSE}(a, b) = \log(c + 1) - \frac{c}{c+1} \log c.$$

Because M assigns higher weights to same-class nodes, $(M\mathbf{x})_1 > (M\mathbf{x})_2$ whenever $x_1 > x_2$. After post-modulation $\mathbf{y} = M\mathbf{x} \odot \mathbf{x}$,

$$\frac{y_1}{y_2} = \frac{(M\mathbf{x})_1 x_1}{(M\mathbf{x})_2 x_2} > \frac{x_1}{x_2} = c > 1.$$

Let $h(x) = \log(x+1) - \frac{x}{x+1} \log x$, with derivative $h'(x) = -\frac{\log x}{(x+1)^2} < 0$ for $x > 1$. Since $h(x)$ is strictly decreasing, enlarging the ratio y_1/y_2 decreases entropy:

$$\text{PSE}(y_1, y_2) < \text{PSE}(x_1, x_2).$$

The result extends to higher dimensions by coordinate-wise analysis, proving that node-wise post-modulation reduces feature entropy. $\square$

C Additional Results

Tab. 6 and Fig. 8–9 provide additional results validating the effectiveness and efficiency of GraphTARIF. As shown in Tab. 6, GraphTARIF achieves superior performance on two heterophilic graphs with substantially fewer parameters than Polynormer and Expformer. Fig. 8 visualizes the first-layer attention features on the Photo dataset, where our method produces representations with higher rank and lower entropy. The t-SNE [34] projection in Fig. 9 further reveals clearer decision boundaries and greater inter-class separability.

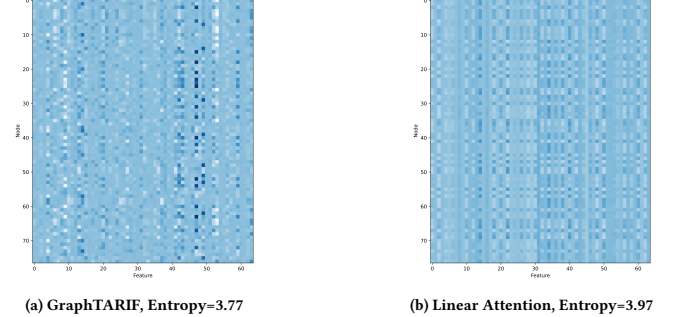


Figure 8: Visualization of the output features on the Photo dataset with entropy analysis.

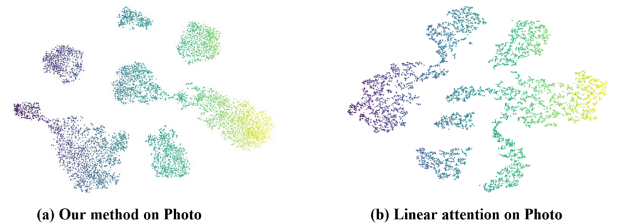


Figure 9: t-SNE visualizations of output features.